

Максимальный элемент массива

Дано: ввести массив целого типа. Найти значение максимального элемента, его индекс, его номер и если их несколько, равных максимальному, то их количество.

Значения эл массива вводим вручную через cin.

Анализ задачи:

1. Резервируем место под 50 эл массива, i1 - номер (=1), s - сумма (=1)
2. Вводим эл массива
for (int i = 0; i < 50; i++) {
 cin << a[i];
}
3. Считаем, что максимальным будет первый элемент
4. В цикле for перебираем эл массива начиная со 2 и сравниваем с максимальным
5. if (a[i] > max) {
 max = a[i];
 i1 = i;
 s = 1;
}
else {
 if (a[i] == max){
 s += 1;
 }
}
6. Выводим max, i1+1, s

Код:

```
#include <iostream>
```

```
#include <locale>
```

```
using namespace std;
```

```
int main() {
```

```
    setlocale(LC_ALL, "ru");
```

```
    int a[50];
```

```
    int n;
```

```
    cout << "Введите количество элементов: ";
```

```
    cin >> n;
```

```
    cout << "Введите " << n << " целых чисел:" << endl;
```

```
    for (int i = 0; i < n; i++) {
```

```
        cin >> a[i];
```

```
    }
```

```
    int max = a[0];
```

```
    int i1 = 0;
```

```
    int s = 1;
```

```

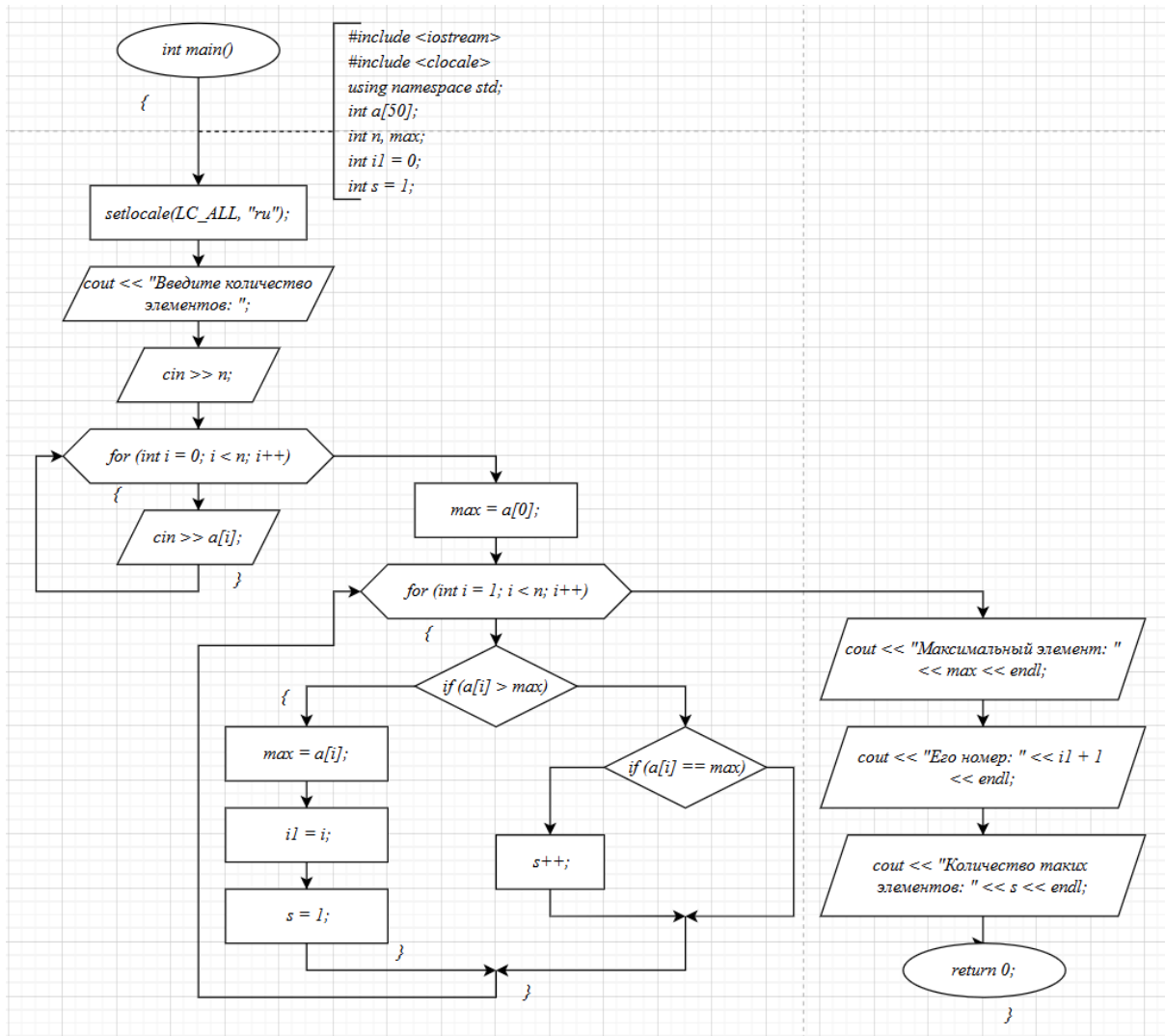
for (int i = 1; i < n; i++) {
    if (a[i] > max) {
        max = a[i];
        i1 = i;
        s = 1;
    }
    else if (a[i] == max) {
        s++;
    }
}

cout << "Максимальный элемент: " << max << endl;
cout << "Его номер: " << i1 + 1 << endl;
cout << "Количество таких элементов: " << s << endl;

return 0;
}

```

Блок-схема:



Перевернуть массив

Дано: одномерный массив целых чисел. Перевернуть массив (поменять местами 1 и последний, 2 и предпоследний и т.д.)

$a[] = \{8, 8, 6, 4, 8, 2\} \rightarrow a[] = \{2, 8, 4, 6, 8, 8\}$

Анализ задачи:

1. Найти формулу симметричных элементов
0 $\rightarrow$ n-1
1 $\rightarrow$ n-2
2 $\rightarrow$ n-3
симметричный эл находится по формуле n-i-1
2. Вводим эл массива в цикле
3. Делим массив пополам
4. Для каждого эл до середины применяем формулу симметричных элементов
5. Если кол-во эл нечетное, то средний эл оставляем без изменений

Код:

```
#include <iostream>
#include <locale>
using namespace std;

int main() {
    setlocale(LC_ALL, "ru");
    int a[50];
    int n;

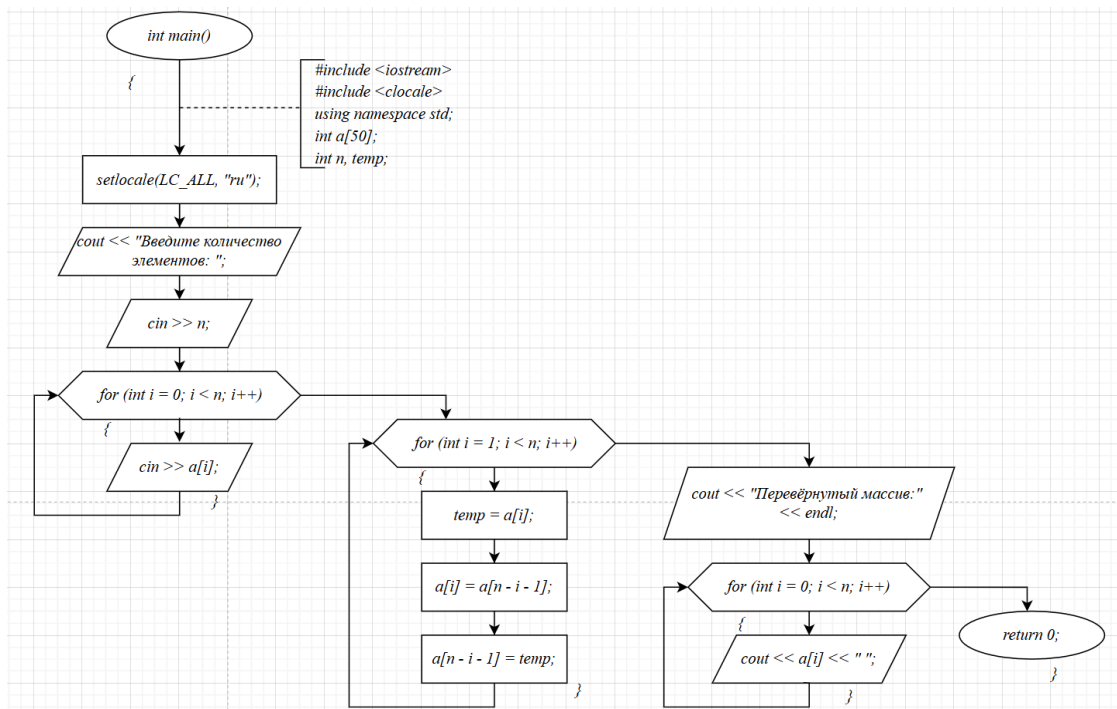
    cout << "Введите количество элементов: ";
    cin >> n;

    cout << "Введите " << n << " целых чисел:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    for (int i = 0; i < n / 2; i++) {
        int temp = a[i];
        a[i] = a[n - i - 1];
        a[n - i - 1] = temp;
    }

    cout << "Перевернутый массив:" << endl;
    for (int i = 0; i < n; i++) {
        cout << a[i] << " ";
    }
    return 0;
}
```

Блок-схема (ошибка во 2м цикле, надо `for (int i = 0; i < n / 2; i++)`):



Ханойская башня

Дано: n - высота башни

Понятие:

Ханойская башня представляет собой перестановочную задачу. Есть три стержня, на один из которых нанизаны кольца различного диаметра в виде пирамиды. Основная цель заключается в перемещении данной пирамиды из колец на другой стержень.

Перемещение дисков должно осуществляться согласно следующим правилам:

1. За один ход можно переместить только один диск.
2. Диски можно перемещать только в порядке убывания их диаметров, то есть меньший диск всегда должен быть размещен на больший. Также диск любого диаметра может быть перемещен на пустой стержень.

Необходимо реализовать программу, которая определяет перестановки этих дисков с помощью наименьшего количества ходов.

Алгоритм решения задачи:

1. Перекладывается $n - 1$ дисков на свободный стержень.
2. Перекладывается n -й диск на нужный стержень.
3. Перекладывается $n - 1$ дисков на нужный стержень.

Чтобы переложить $n - 1$ дисков:

1. Перекладывается $n - 2$ дисков на свободный стержень.
2. Перекладывается $(n - 1)$ -й диск на нужный стержень.
3. Перекладывается $n - 2$ дисков на нужный стержень.

и т.д.

Код:

```
#include <iostream>
```

```
#include <locale>
```

```
using namespace std;
```

```
int N;
```

```
int tower1[10], h1 = 0;
```

```
int tower2[10], h2 = 0;
```

```
int tower3[10], h3 = 0;
```

```
int moveCount = 0;
```

```
void printTowers() {
```

```
    cout << "\n";
```

```
    for (int level = N - 1; level >= 0; level--) {
```

```
        // Стержень 1
```

```
        if (level < h1) {
```

```
            int disk = tower1[level];
```

```
            for (int s = 0; s < N - disk; s++) cout << " ";
```

```
            for (int d = 0; d < 2 * disk - 1; d++) cout << "=";
```

```
            for (int s = 0; s < N - disk; s++) cout << " ";
```

```
        }
```

```
        else {
```

```
            for (int s = 0; s < N - 1; s++) cout << " ";
```

```
            cout << "|";
```

```
            for (int s = 0; s < N - 1; s++) cout << " ";
```

```

    }
    cout << " ";

    // Стержень 2
    if (level < h2) {
        int disk = tower2[level];
        for (int s = 0; s < N - disk; s++) cout << " ";
        for (int d = 0; d < 2 * disk - 1; d++) cout << "=";
        for (int s = 0; s < N - disk; s++) cout << " ";
    }
    else {
        for (int s = 0; s < N - 1; s++) cout << " ";
        cout << "|";
        for (int s = 0; s < N - 1; s++) cout << " ";
    }
    cout << " ";

    // Стержень 3
    if (level < h3) {
        int disk = tower3[level];
        for (int s = 0; s < N - disk; s++) cout << " ";
        for (int d = 0; d < 2 * disk - 1; d++) cout << "=";
        for (int s = 0; s < N - disk; s++) cout << " ";
    }
    else {
        for (int s = 0; s < N - 1; s++) cout << " ";
        cout << "|";
        for (int s = 0; s < N - 1; s++) cout << " ";
    }
    cout << endl;
}
cout << "-----" << endl;
}

void moveDisk(int from, int to) {
    int disk;

    if (from == 1) {
        disk = tower1[--h1];
    }
    else if (from == 2) {
        disk = tower2[--h2];
    }
    else {
        disk = tower3[--h3];
    }

    if (to == 1) {
        tower1[h1++] = disk;
    }
    else if (to == 2) {
        tower2[h2++] = disk;
    }
    else {
        tower3[h3++] = disk;
    }
}

```

```

    }

    moveCount++;
    cout << "Ход " << moveCount << ": " << from << " => " << to << endl;
    printTowers();
}

void han(int n, int start, int point, int temp) {
    if (n <= 0)
        return;

    han(n - 1, start, temp, point);
    moveDisk(start, point);
    han(n - 1, temp, point, start);
}

int main() {
    setlocale(LC_ALL, "ru");

    int n;
    cout << "Введите количество дисков: ";
    cin >> n;
    N = n;

    for (int i = 0; i < N; i++) {
        tower1[i] = N - i;
    }
    h1 = N;

    cout << "\nНачальное состояние:" << endl;
    printTowers();

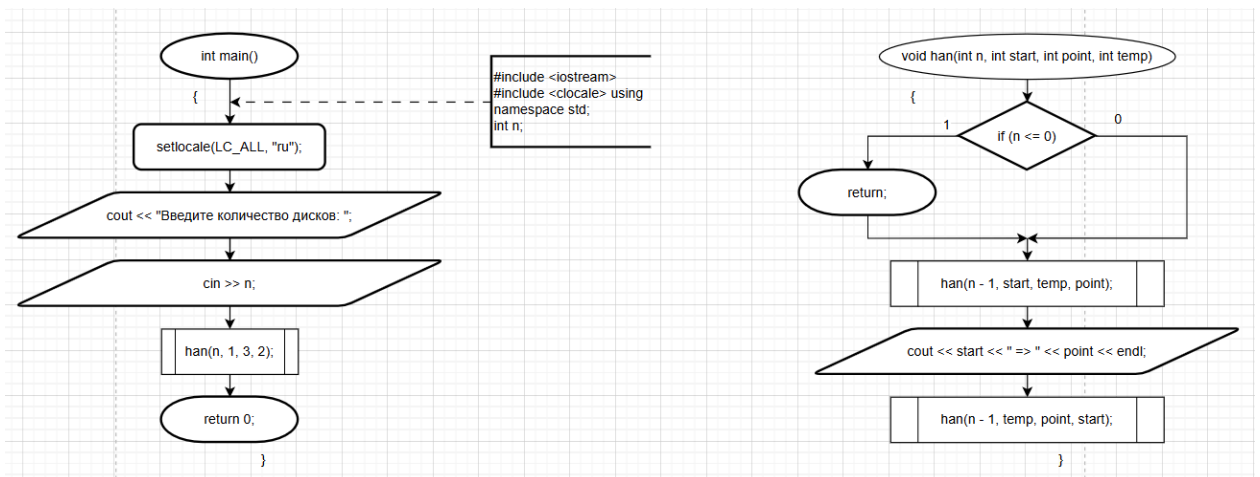
    han(n, 1, 3, 2);

    cout << "Башня собрана! Всего ходов: " << moveCount << endl;

    return 0;
}

```

Блок-схема:



Визуализация:

```
Введите количество дисков: 3

Начальное состояние:

  =      |      |
  ==     |      |
  =====|      |
-----

Ход 1: 1 => 3

  |      |      |
  ==     |      |
  =====|      =
-----

Ход 2: 1 => 2

  |      |      |
  =====|      =
-----

Ход 3: 3 => 2

  |      |      |
  =====|      =
-----

Ход 4: 1 => 3

  |      |      |
  =====|      =
-----

Ход 5: 2 => 1

  |      |      |
  =      ==     =====
-----

Ход 6: 2 => 3

  |      |      |
  =      |      ==
  =====|
-----

Ход 7: 1 => 3

  |      |      =
  =====|
-----

Башня собрана! Всего ходов: 7
```

Числа Фибоначчи

Дано: n - длина последовательности чисел Фибоначчи

Алгоритм решения задачи:

Числа Фибоначчи – это ряд, состоящий из целых чисел. Их особенность заключается в том, что каждый элемент представляет собой сумму двух предыдущих чисел (кроме первого и второго числа).

Последовательность Фибоначчи начинается с 0 и 1: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...

Алгоритм рекурсивной функции:

1. Если $n = 0$, вернуть 0.
2. Если $n = 1$, вернуть 1.
3. В противном случае вернуть сумму двух предыдущих чисел $F(n-1) + F(n-2)$.

Код:

```
#include <iostream>
#include <locale>
using namespace std;

int fib(int n) {
    if (n == 1)
        return 0;
    if (n == 2)
        return 1;
    return fib(n - 2) + fib(n - 1);
}

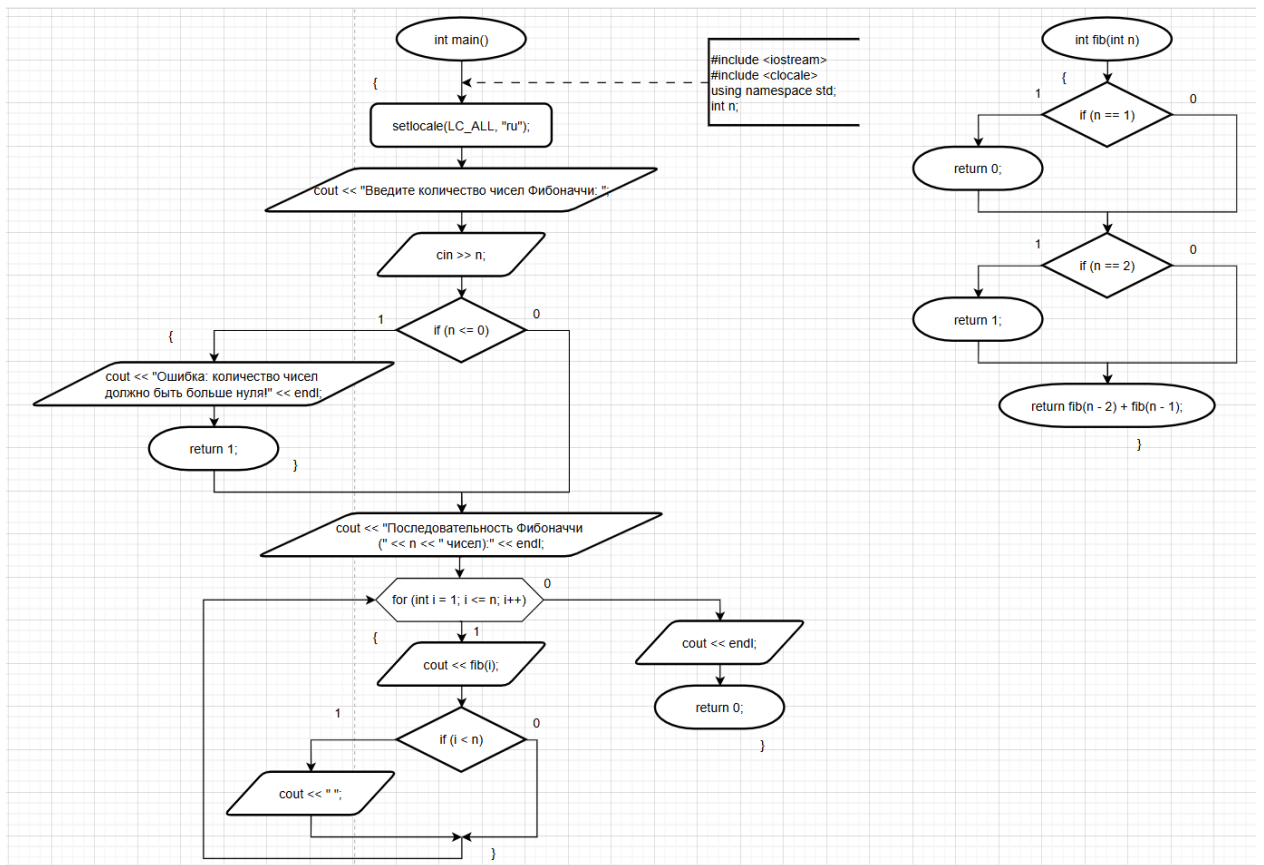
int main() {
    setlocale(LC_ALL, "ru");

    int n;
    cout << "Введите количество чисел Фибоначчи: ";
    cin >> n;

    if (n <= 0) {
        cout << "Ошибка: количество чисел должно быть больше нуля!" << endl;
        return 1;
    }

    cout << "Последовательность Фибоначчи (" << n << " чисел):" << endl;
    for (int i = 1; i <= n; i++) {
        cout << fib(i);
        if (i < n) {
            cout << " ";
        }
    }
    cout << endl;
    return 0;
}
```

Блок-схема:



О восьми Ферзях

Дано: Необходимо разработать алгоритм размещения восьми ферзей на шахматной доске размером 8x8 таким образом, чтобы исключить возможность взаимного нападения между ними.

Алгоритм решения задачи:

1. Размещение первого ферзя в клетке, расположенной в первой строке и первом столбце.
2. Размещение ферзя в следующей строке таким образом, чтобы он не находился под атакой уже размещенных ферзей.
3. В случае невозможности размещения ферзя в текущей строке без нарушения условий задачи, необходимо вернуться к предыдущей строке и выбрать следующую допустимую клетку для размещения ферзя.
4. Повторение шагов 2 и 3 до размещения всех восьми ферзей на шахматной доске.

Код:

```
#include <iostream>
using namespace std;

const int size_arr = 8;
int arr[size_arr][size_arr] = { 0 };
bool solutionFound = false; // Флаг для остановки после первого решения

bool check(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (arr[i][col]) return false;
    }

    for (int i = 1; row - i >= 0 && col - i >= 0; i++) {
        if (arr[row - i][col - i]) return false;
    }

    for (int i = 1; row - i >= 0 && col + i < size_arr; i++) {
        if (arr[row - i][col + i]) return false;
    }

    return true;
}

void show() {
    cout << " a b c d e f g h\n";
    cout << " +-----+\n";

    for (int i = 0; i < size_arr; i++) {
        cout << i + 1 << " |";
        for (int j = 0; j < size_arr; j++) {
```

```

        cout << (arr[i][j] ? "Ф" : ".") << " ";
    }
    cout << "\n";
}

cout << " +-----+\n";
}

void game(int row, int step) {
    if (solutionFound) return; // Если решение уже найдено, выходим

    if (row == size_arr) {
        show();
        solutionFound = true;
        return;
    }

    for (int col = 0; col < size_arr; col++) {
        if (solutionFound) return; // Проверка после каждого шага

        if (check(row, col)) {
            arr[row][col] = 1;

            // Выводим каждый шаг установки ферзя
            cout << "\n Шаг " << step + 1 << ": Ставим ферзя в позицию "
                << char('a' + col) << (row + 1) << "\n";
            show();

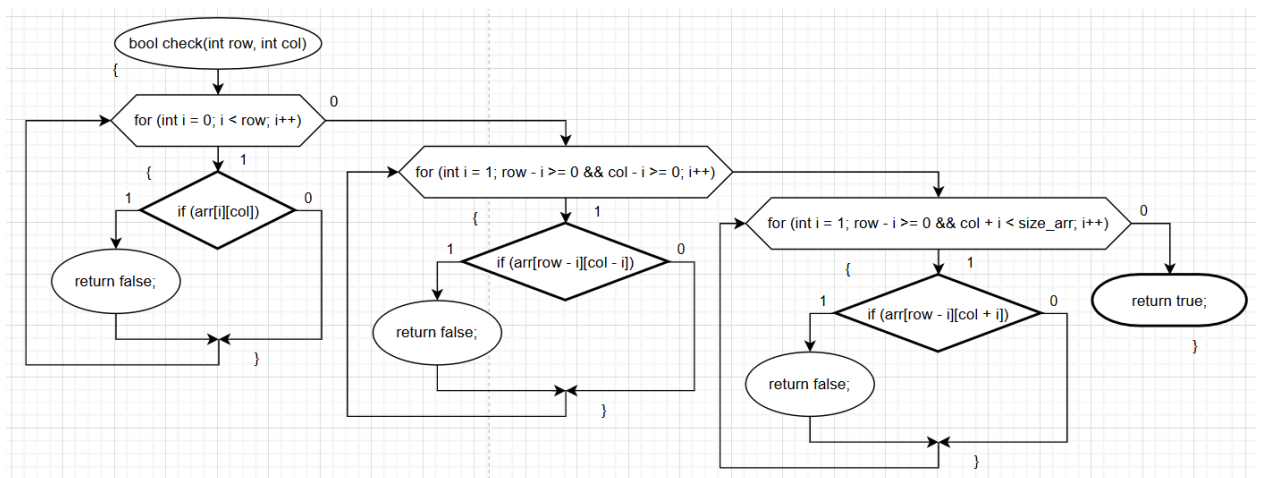
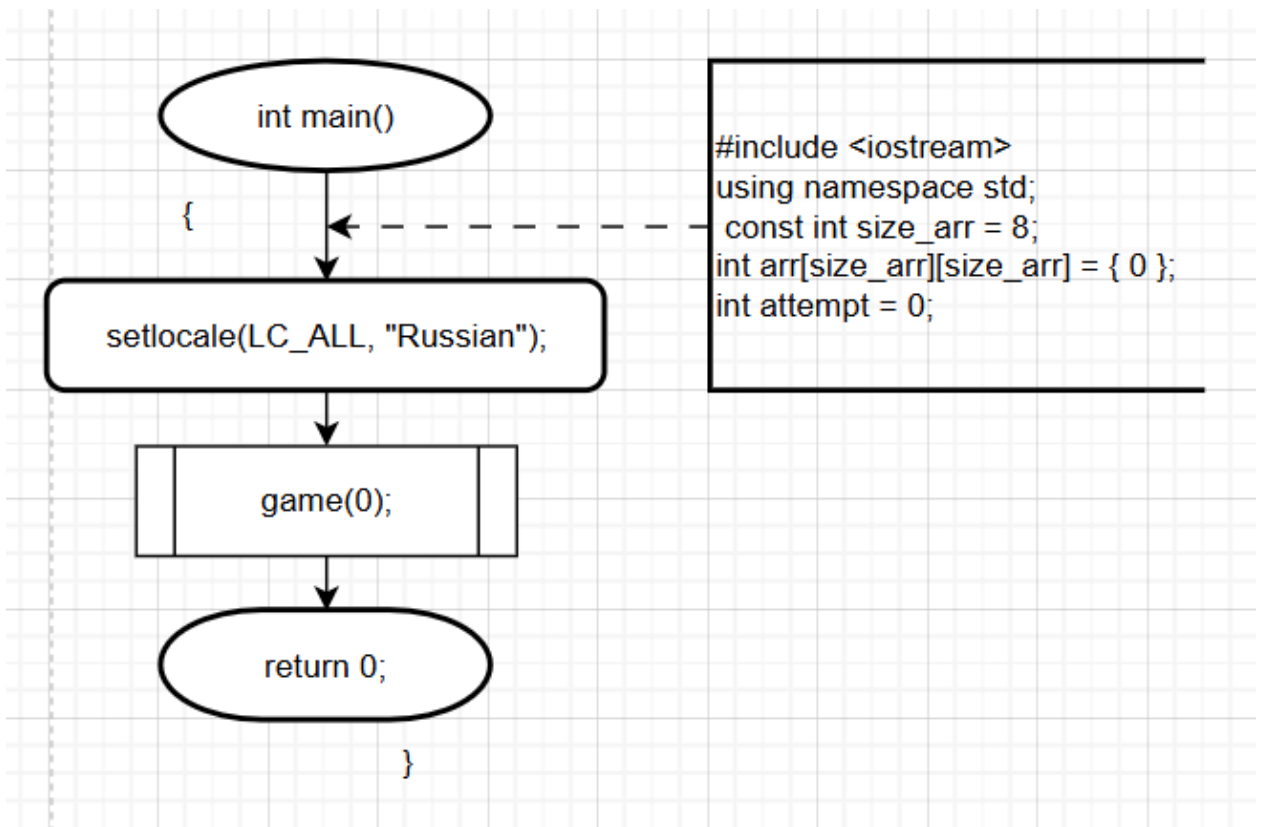
            game(row + 1, step + 1);

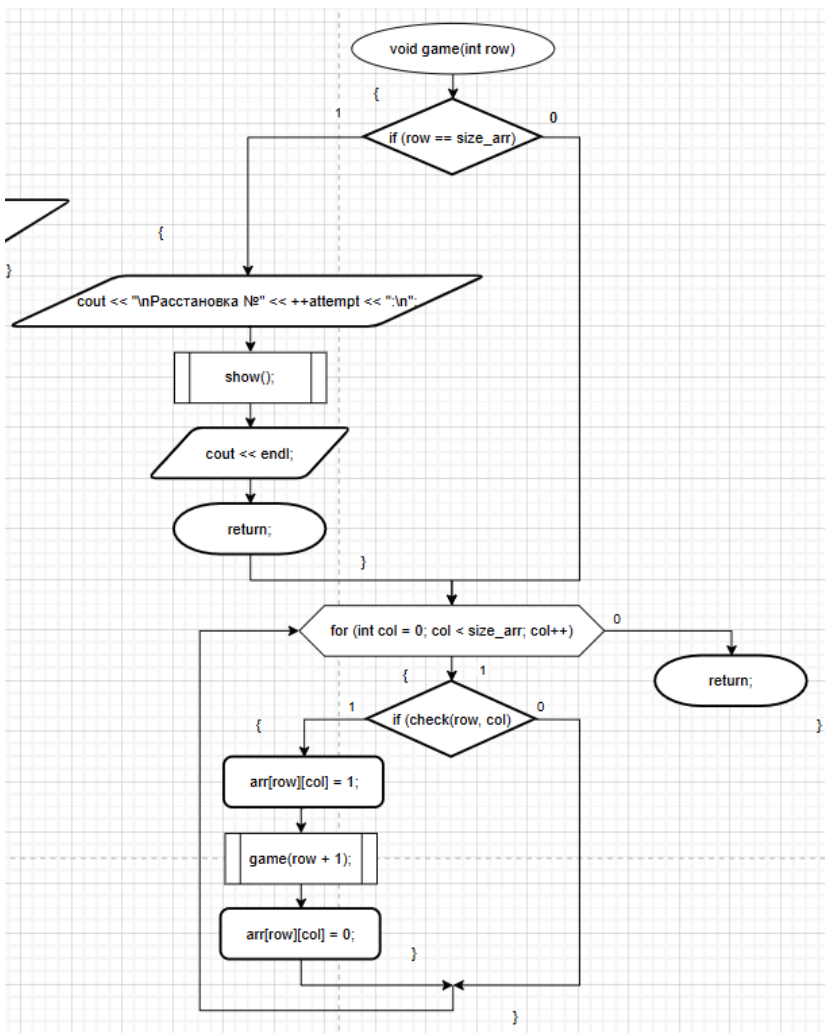
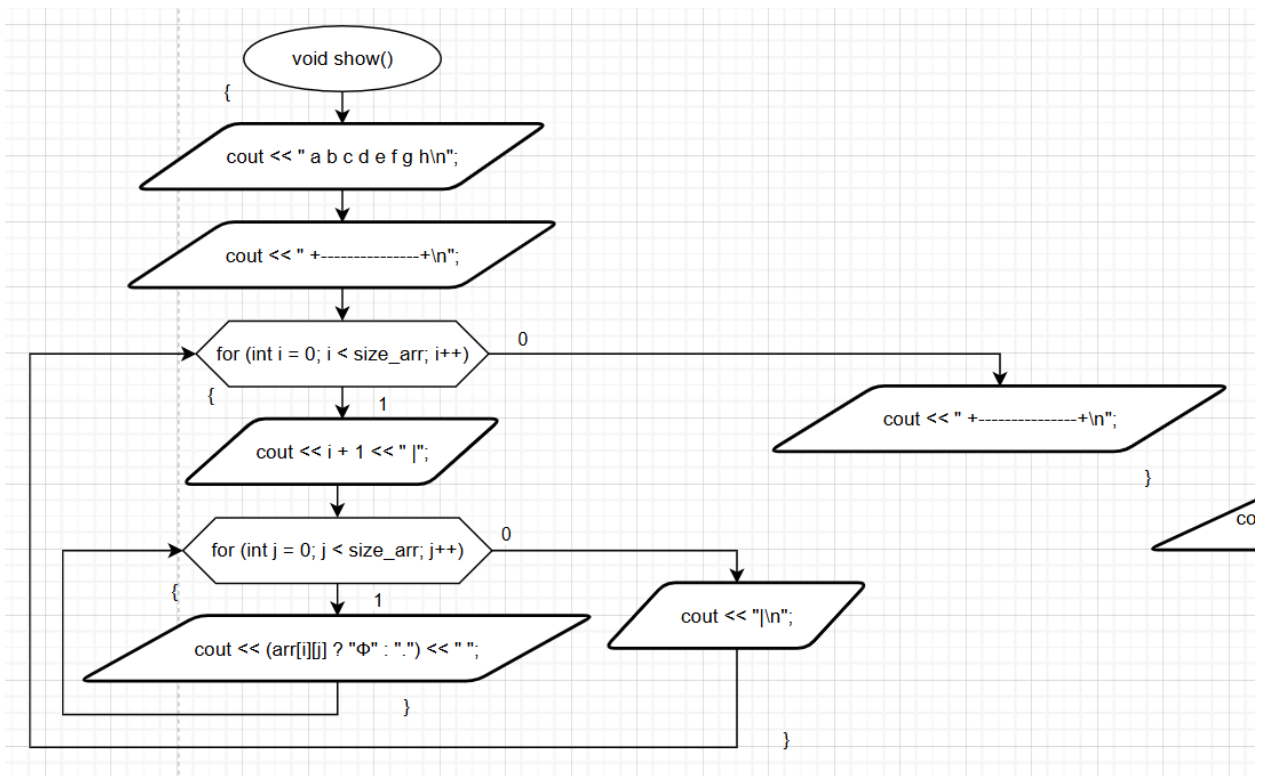
            if (!solutionFound) {
                // Если решение не найдено, показываем удаление ферзя
                arr[row][col] = 0;
                cout << "\n Откат: убираем ферзя с позиции "
                    << char('a' + col) << (row + 1) << "\n";
                show();
            }
        }
    }
}

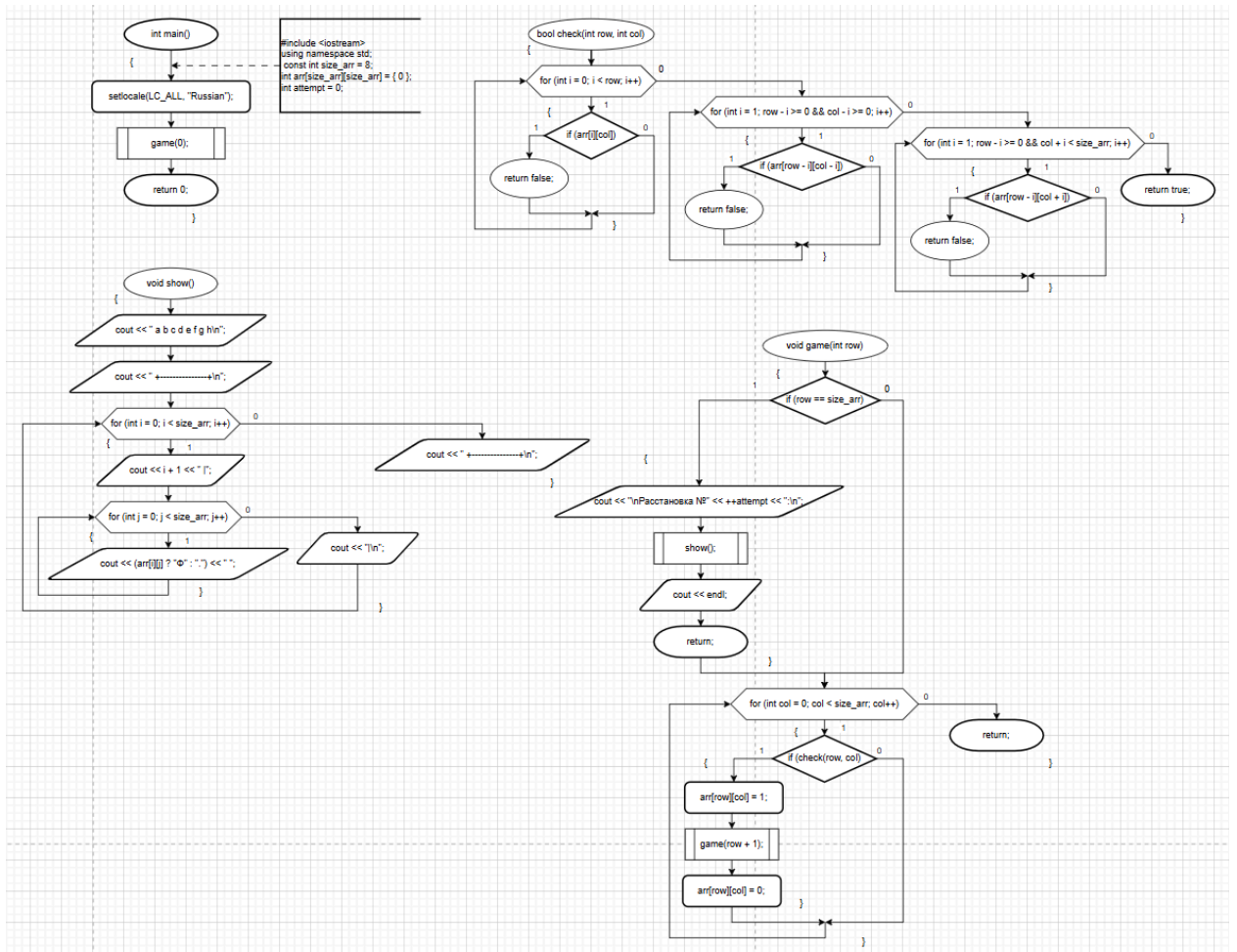
int main() {
    setlocale(LC_ALL, "Russian");
    game(0, 0);
    return 0;
}

```

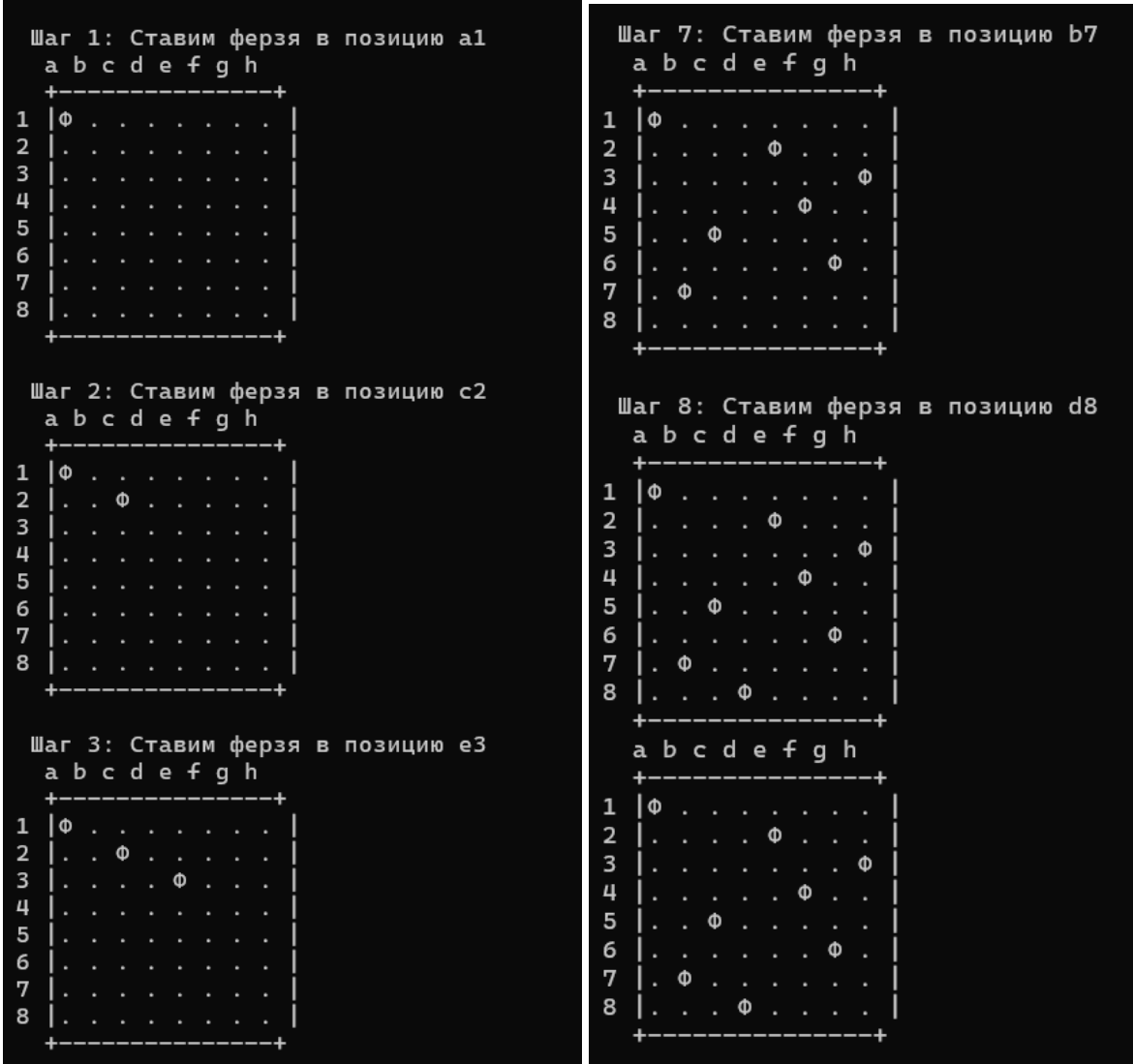
Блок-схема:







Визуализация:



Скобочки

Дано: Строка, вида: “ ({ [] }) { } ([])”. Требуется проверить, расположены ли скобки в ней верно.

Алгоритм решения задачи:

1. Создаем пустой стек.
2. Просматриваем все элементы строки по порядку
3. Если встречена открывающая скобка, то заносим её в стек.
4. Если символ – закрывающая скобка, проверяем вершину стека. На вершине стека, должна быть соответствующая открывающая скобка (если условие ложно, значит выражение неправильное)
5. Если после прохода, стек не пуст, значит выражение неправильное.

Код:

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
template <class T>
```

```
struct Node {
```

```
    T data;
```

```
    Node<T>* next_ptr;
```

```
};
```

```
template <class T>
```

```
struct Stack {
```

```
    Node<T>* head;
```

```
    int size;
```

```
};
```

```
template <class T>
```

```
void SetNode(Node<T>*& node, T data = T(), Node<T>* next_ptr = nullptr) {
```

```
    node = new Node<T>();
```

```
    node->next_ptr = next_ptr;
```

```
    node->data = data;
```

```
}
```

```
template <class T>
```

```
void SetStack(Stack<T>& tmp) {
```

```
    tmp.head = nullptr;
```

```
    tmp.size = 0;
```

```
}
```

```
template <class T>
```

```

void Push(Stack<T>& tmp, const T& data) {
    Node<T>* new_node;
    SetNode(new_node, data, tmp.head);
    tmp.head = new_node;
    tmp.size++;
}

```

```

template <class T>
void Pop(Stack<T>& tmp) {
    if (tmp.head != nullptr) {
        Node<T>* delete_element = tmp.head;
        tmp.head = tmp.head->next_ptr;
        delete delete_element;
        tmp.size--;
    }
}

```

```

// Функция для проверки соответствия скобок
bool IsMatchingBracket(char open, char close) {
    switch (open) {
        case '(': return close == ')';
        case '{': return close == '}';
        case '[': return close == ']';
        default: return false;
    }
}

```

```

// Основная функция проверки строки
bool CheckBrackets(const string& str) {
    Stack<char> bracketStack;
    SetStack(bracketStack);

    for (char ch : str) {
        if (ch == '(' || ch == '{' || ch == '[') {
            Push(bracketStack, ch); // Добавляем открывающую скобку в стек
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (bracketStack.head == nullptr) {
                return false; // Нет соответствующей открывающей скобки
            }
            char topBracket = bracketStack.head->data;
            if (!IsMatchingBracket(topBracket, ch)) {
                return false; // Не соответствует
            }
            Pop(bracketStack); // Удаляем открывающую скобку из стека
        }
    }
}

```

```

    }

    return bracketStack.head == nullptr;
}

int main() {
    setlocale(LC_ALL, "ru");

    string testString = "( { [ ] } ) { } ( [ ] )";

    if (CheckBrackets(testString)) {
        cout << "Скобки расставлены корректно!" << endl;
    }
    else {
        cout << "Ошибка в расстановке скобок!" << endl;
    }

    return 0;
}

```

Результат выполнения программы:

